

Agent eBPF Filter

面向 AI Agent 的 内核级运行时安全与可观测平台

不是罗列更多功能，而是把一个关键问题做深：
Agent 的语义承诺，如何在操作系统事实里被验证和约束。

内核事实采集

LSM/cgroup 前置阻断

语义 - 事实一致性

负责人 / 学院 / 联系方式：
【待填】
推荐学院：【待填】
负责人：【待填】
手机号 / QQ：【待填】

技术深度与应用前景版 · 2026-05-26

核心主张

从“看得到”升级到“拦得住、讲得清、能落地”

4 层

采集、归因、判定、阻断

3 类

高校 / 团队 / 企业落地场景

1 条

可复盘 Agent 证据链

路演结构：少讲技术广度，深讲一条闭环

前半段证明为什么必须做，主体部分讲清技术深水区，后半段说明能在哪些场景形成价值。

- 01 为什么是刚需** AI Agent 使用率、密钥泄露和 Copilot/MCP 案例说明运行时边界已经成为现实问题。
- 02 深水区一：内核事实** 围绕 exec/file/net/fork 采集、ringbuf、map、上下文字段，而不是泛泛讲“监控”。
- 03 深水区二：前置阻断** BPF LSM 和 cgroup 把策略放到危险动作发生前，形成确定性拒绝证据。
- 04 深水区三：语义归因** 把 run/task/tool_call 与 PID、PPID、cwd、cgroup、路径、网络端点连成证据链。
- 05 应用前景** 高校实验室、AI coding 团队、企业私有化和教育服务，分别给出落地价值。

答辩策略：避免“我们还做了 X/Y/Z”的横向铺陈，改为反复回答“为什么这个内核闭环难、为什么能落地”。

数据证据：Agent 普及越快，运行时事实越稀缺

公开调研和安全报告共同指向同一个缺口：AI 工具进入执行链，但审计和控制仍停留在外围。

84%

Stack Overflow 2025：正在使用或计划使用 AI 开发工具 [1]

28.65M

GitGuardian 2026：2025 年 public GitHub 新增硬编码密钥 [2]

\$4.44M

IBM 2025：全球平均数据泄露成本 [3]

97%

IBM 2025：AI 相关事件组织缺少适当访问控制 [3]

24,008

GitGuardian 2026：公开 MCP 配置中暴露的唯一密钥 [2]

Sentiment and usage / All Respondents

AI tools in the development process



2025
Developer
Survey

Source: survey.stackoverflow.co/2025
Data licensed under Open Database License (ODbL)

采用率提升不是重点，重点是 Agent 已经开始触及终端、文件和网络行为。安全边界必须落到运行时 [1]。



软件生产提速后，密钥、依赖脚本、MCP 配置的扩散速度也变快；只看 prompt 已经不够 [2]。

真实案例：风险不是模型幻觉，而是执行链越界

报告截图保留为外部证据，但结论聚焦到一个工程问题：如何把越界行为在 OS 层归因和拦截。



上下文越权 [5][6]

Copilot / EchoLeak 类案例说明：外部输入可能触发内部知识检索或数据泄露，风险发生在工具链边界。

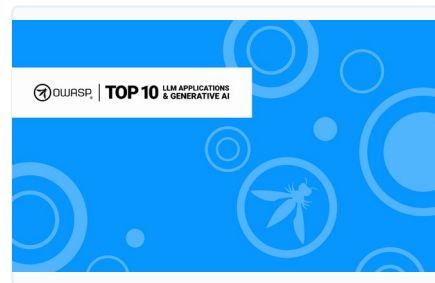
归因 + 拦截 + 回放



配置即攻击面 [2]

MCP 配置中的有效密钥进入公共仓库，说明 Agent 生态的本机配置、工具描述和凭据都需要治理。

归因 + 拦截 + 回放



工具误用 [4]

OWASP 将 tool misuse、excessive agency、untraceability 列为重点，正对应本项目的运行时证据链。

归因 + 拦截 + 回放

核心问题：不是“多做几个监控项”，而是补上证据链断点

技术深度集中在一个闭环：Agent 声称要做什么、系统实际做了什么、危险动作是否被前置拦截。

断点一：意图到进程

Agent 的 `run/task/tool_call` 通常进入 `shell`，再派生 `python/node/git/npm/curl`。传统日志看到 `PID`，却丢失了意图来源。

断点二：进程到资源

危险行为落在 `open/connect/sendmsg/rename` 等 OS 事件上，必须记录路径、端点、`cwd`、`cgroup` 和父子关系。

断点三：告警到阻断

事后告警无法证明动作没有发生。对高风险路径和外联，需要在 `LSM/cgroup` 决策点返回拒绝。

因此本项目不追求“覆盖所有安全功能”，而是把 Agent 运行时安全最难的一段——语义归因 + 内核事实 + 前置阻断——做深。

总体架构：四层闭环，每层只解决一个深问题

从语义入口到内核决策再到证据回放，架构强调深度链路而不是横向功能堆叠。

语义入口

wrapper / native hooks / adapters 捕获 run、task、tool_call、intent



上下文继承

agent_run_id / task_id / tool_call_id / trace_id 随 PID、PPID、cgroup 传播



内核事实

eBPF 采集 exec、file、net、fork，形成不可伪造的 OS 行为证据



前置处置

BPF LSM 与 cgroup 在动作发生前做 ALLOW / ALERT / BLOCK / REWRITE

可答辩表达：我们不是做一个“安全看板”，而是把 Agent 意图和内核事实接成一条可验证、可阻断、可回放的闭环。

技术深水区一： eBPF 事实采集不是日志替代，而是可信证据底座

重点解释为什么选 eBPF：低侵入、内核态事件源、可关联进程树和资源对象。

采集点

execve / openat / connect / sendto /
recvfrom / bind / unlink / mkdir / ioctl 等关
键 syscall 与网络路径。

数据路径

BPF map 维护 Agent 进程、关注命令与关
注路径；ringbuf 将事件送到用户态
runtime。

证据字段

PID/TGID/PPID、comm、cwd、路径、网络端点、
cgroup_id、时间戳、策略结果，统一进入
EventEnvelope。

内核事件采集模块
事件归一化与分发模块
运行时状态管理模块
证据信封模型模块

技术难点

既要拿到足够细的事实字段，又不能把每个 syscall 都变成
高开销日志；未来可扩展为可配置采集策略和行业模板。

技术深水区二： BPF LSM + cgroup 把安全决策前置到内核路径

真正的亮点不是“发现风险”，而是在危险动作发生前给出确定性拒绝。

文件 / 执行

bprm_check_security、file_open、file_permission、inode_*：在执行、读写、创建、删除、重命名前判断策略。

网络外联

connect4/connect6、sendmsg4/sendmsg6：按 cgroup、IP、IPv6、端口对 TCP/UDP 外联做前置约束。

决策原则

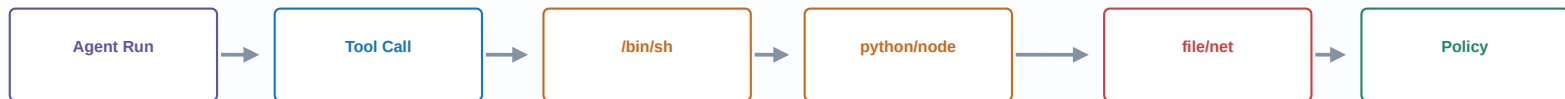
内核态只做确定性 map lookup；ML/LLM 只输出建议，不直接进入内核阻断路径。

答辩亮点

评委能听懂的差异：UI 标红只是提醒，LSM/cgroup 返回 -EACCES 或拒绝 connect 才是 OS 层真实拦截。

技术深水区三：进程上下文继承解决 Agent 子进程“失踪”

真实风险常发生在 shell/python/node/git/npm/curl 子进程，必须把语义上下文随进程树传播。



A 继承字段

agent_run_id、task_id、tool_call_id、trace_id 从父 PID 或 cgroup 继承，避免只看到孤立子进程。

B 事实字段

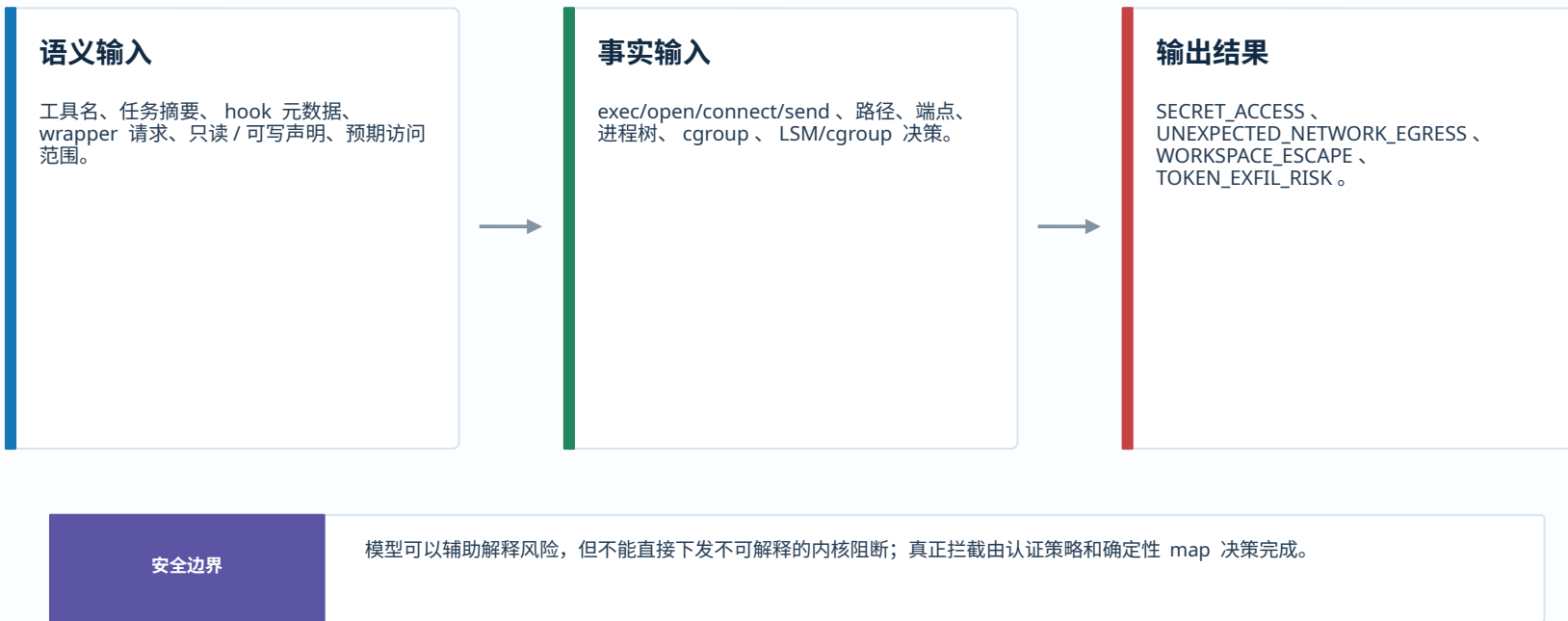
PID/TGID/PPID、comm、cwd、cgroup_id 记录真实执行链，支撑后续图谱回放。

C 归因结果

把 file/net/policy 事件归到具体 Agent run 和 tool call，而不是只归到 python 或 curl。

技术深水区四：语义 - 事实一致性，不靠模型猜测做最终裁决

让 LLM/ 规则负责解释和建议，让内核事实负责证据，让确定性策略负责阻断。



工程深度：性能、权限和误报边界必须讲清楚

高权限安全工具不能只展示能力，还要说明开销、权限、误报和隐私边界如何控制。

性能边界

只采关键 syscall 与网络路径；tracked_comm/path、agent_pids、cgroup 过滤降低噪声；ringbuf 异步上报避免阻塞正常路径。

权限边界

eBPF 加载需要特权；配置 API 使用 token；shell 与策略变更有 runtime gate；TLS 明文捕获默认关闭。

误报边界

先观察后阻断；高风险路径默认告警，确认后进入 LSM/cgroup 阻断；ML/LLM 输出进入人工确认或策略模板。

合规边界

用于本地 / 授权环境；事件流以摘要、路径、端点、长度、角色为主；演示数据必须脱敏。

这一页用于回应评委追问：能不能跑、会不会拖慢、权限是否过大、误报怎么处理。

验证方式：用三类实验证明深度闭环，而不是只放 UI 截图

现场演示围绕“采集到、归因准、拦得住”三件事设计。

实验 A

敏感路径读取

Agent 声称只读项目文件，但子进程访问 .env / ssh key → SECRET_ACCESS + 任务归因

实验 B

异常外联

curl/python 子进程连接未知 IP:port → UNEXPECTED_NETWORK_EGRESS，并关联到 tool_call

实验 C

前置阻断

命中 LSM/cgroup 策略后返回 EACCES 或拒绝 connect/sendmsg，证据进入图谱

实验 D

图谱回放

Agent Run → Tool Call → Process → File/Network → Policy，形成可提交证据链

验收指标

检测率 / 阻断率、误报样例、平均事件延迟、CPU/ 内存开销、证据链完整率。

未来前景总览：从单机工具走向 Agent 安全基础设施

应用前景不再泛泛讲市场，而是把内核级证据链扩展成教育、团队、企业三类长期基础能力。

高校实验室

教学 / 科研 / 竞赛

沉淀为 AI 安全与系统安全实验平台，支撑课程、论文、软著、竞赛和校企联合实践。

价值

成果转化

AI coding 团队

开发治理

发展为团队级 Agent 安全工作台，持续管理敏感文件、依赖脚本、MCP 工具和异常外联。

价值

降低事故

企业安全

合规与私有化

演进为企业 Agent runtime security 基础设施，接入审计、SIEM/OTLP、策略审批和合规留存。

价值

合规落地

应用前景一：高校实验室，从竞赛项目变成教学与科研平台

国创赛场景下，最容易先落地的是可复现实验、课程实践和安全竞赛训练。

1 课程实验

eBPF syscall 观测、LSM 阻断、Agent 风险 replay，可做成 AI 安全 / 系统安全课程实验。

2 科研题目

Agentic AI runtime guardrail、语义 - 事实一致性、执行图谱和误报控制，可沉淀论文 / 专利方向。

3 竞赛训练

用良性 / 恶意任务 replay 构造靶场，训练学生识别 prompt 注入、MCP 密钥、异常外联。

落地路径

先以开源 demo + 实验文档进入课程 / 实验室；用竞赛演示和测试报告证明可复现；再争取软著、导师课题、校企联合实践。

近期成果

演示视频、实验指导书、benchmark 数据集、软著 / 专利材料、导师推荐或试点证明。

应用前景二： AI coding 团队，解决“能用但不敢放开用”

团队不是缺 AI 工具，而是缺本地执行链的边界、审计和复盘能力。



付费理由

AI-assisted commits 与 MCP 配置泄露会直接影响代码资产、凭据和客户数据；团队负责人需要可追责证据。

策略模板

只读审查、依赖安装、网络访问、敏感路径等团队级策略包

审计报表

按 `run/task/tool_call` 输出证据链，支持复盘与责任界定

训练样本

将真实告警转成 `replay` 数据，持续降低误报和漏报

商业入口

Team Pro：团队策略、多人审计、报表导出、告警训练集。

未来前景三：企业私有化，成为 Agent 安全基础设施

企业价值不只是 UI，而是把 Agent 执行证据送进合规、审计、安全运营和策略治理流程。

部署形态

单机开发机
团队网关
K8s / 多节点
离线私有化

集成对象

SIEM / OTLP
审计留存
SSO / token
策略审批

交付价值

外联约束
敏感路径保护
事故复盘
合规报告

服务收入

PoC
私有化部署
策略调优
培训与靶场

长期前景：当 Agent 成为企业标准生产力工具，runtime guardrail 会像日志、EDR、CI 安全一样成为基础设施。

商业模式：围绕深度能力收费，而不是围绕页面数量收费

开源用于建立信任，付费集中在团队治理、私有化交付和教育内容。

版本	价格	交付内容	付费动机
Community	免费 / 开源	单机观测、基础告警、教学实验	开发者获客与课程传播
Team Pro	3.98 万 / 年起	团队策略、报表、训练样本、多用户	AI coding 团队治理
Enterprise	19.8 万 / 年起	私有化、多节点、SSO、SIEM、审计留存	企业合规与安全运营
Education Kit	3 万 / 套起	课程实验、靶场、竞赛训练营	高校实验室与培训
Consulting	5-20 万 / 项目	PoC、红队 replay、策略调优	落地服务收入

注：价格为校赛版测算，提交前建议补充真实访谈、试点意向或导师 / 企业推荐证明。

实施路线：先把深度闭环跑稳，再扩展场景

路线图从“证明内核闭环可靠”开始，而不是先堆平台功能。



阶段判断

如果内核闭环和低误报验证不充分，不急于扩大 UI 功能；先把最难的技术证明做扎实。

团队分工与合规边界：围绕深度闭环组织，而不是按页面分工

提交前将【待填】替换为真实姓名、学号、年级、学院与贡献证据。

技术主线

eBPF/LSM/cgroup：【待填】
上下文继承 / 执行图谱：【待填】
语义风险检测：【待填】
性能与误报验证：【待填】

应用主线

高校实验与课程：【待填】
团队 / 企业访谈：【待填】
商业模式与报价：【待填】
演示视频与材料：【待填】

合规边界

- 只在本地 / 授权环境使用
- 先观察后阻断
- 不采集未授权数据
- TLS 明文捕获默认关闭
- 演示数据必须脱敏

团队证明建议：commit 记录、模块负责人截图、导师指导记录、测试报告、软著 / 论文 / 专利分工。

总结：技术深度解决信任问题，应用前景决定项目价值

用公开数据说明刚需，用内核闭环证明技术深度，用明确场景说明落地前景。

深度 1：可信事实

eBPF 采集关键 OS 行为，EventEnvelope 统一语义和事实字段，形成不可只靠 prompt 替代的证据底座。

深度 2：前置控制

BPF LSM/cgroup 把策略放到危险动作发生前，证明项目不是事后看板，而是 runtime guardrail。

前景：三类场景

高校实验室、AI coding 团队、企业私有化分别对应成果转化、团队治理和合规运营。

需要支持：导师 / 学院资源、试点用户、软著 / 专利指导。目标：校赛晋级、省赛打磨、形成可转化产品。

本 PPT 中统计数据、案例与外部框架已用 [1]–[6] 标注来源；完整引用信息见下一页。

谢谢各位老师，请批评指正

引用信息：统计数据、案例与风险框架来源

答辩时可按编号追溯数据、报告截图和外部安全框架。

[1]

Stack Overflow Developer Survey 2025 — AI tools adoption / usage

<https://survey.stackoverflow.co/2025/ai>

[2]

GitGuardian State of Secrets Sprawl 2026 — secrets scale, MCP secrets, AI-assisted commit risk

<https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/>

[3]

IBM Cost of a Data Breach Report 2025 / IBM newsroom — breach cost and AI access-control gap

<https://www.ibm.com/reports/data-breach>

[4]

OWASP Top 10 for LLM Applications / Agentic AI Threats / MCP Tool Poisoning

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

[5]

Cloud Security Alliance research note — M365 Copilot / CVE-2026-24299 security guidance

<https://labs.cloudsecurityalliance.org/>

[6]

NVD CVE-2025-32711 — EchoLeak vulnerability record

<https://nvd.nist.gov/vuln/detail/CVE-2025-32711>

说明：PPT 内使用短编号标注来源，详细数值以引用页链接和原报告为准；2026 正式竞赛材料提交前建议再次复核访问状态。